

01 QUERY STRUCTURE

Skeleton
SELECT [DISTINCT] expressions
FROM R [JOIN S ON condition]
[WHERE row_condition]
[GROUP BY keys]
[HAVING group_condition]
[ORDER BY expression [ASC|DESC]]
[LIMIT n];

Logical processing order
FROM/JOIN -> WHERE -> GROUP BY
-> HAVING -> SELECT
-> ORDER BY -> LIMIT

! Window ORDER BY controls the window calculation, not the final output order. Add an outer ORDER BY when display order matters.

02 MYSQL SYNTAX

String operations
LIKE '%abc%' -- %: any length; _ : one character
CONCAT(a,b) -- || is logical OR by default in MySQL
UPPER(s); LOWER(s);
CHAR_LENGTH(s); LENGTH(s); -- characters vs bytes
SUBSTRING(s,pos,len);

COND LIKE case sensitivity depends on collation; MySQL is commonly case-insensitive.

! String positions start at 1: SUBSTRING(s,0,1) returns empty; use SUBSTRING(s,1,1) for the first character.

Common MySQL data types
TINYINT; SMALLINT; MEDIUMINT; INT/INTEGER; BIGINT; [UNSIGNED]
DECIMAL(p,s); NUMERIC(p,s);
FLOAT; DOUBLE;
CHAR(n); VARCHAR(n); TEXT;
BINARY(n); VARBINARY(n); BLOB;
DATE; TIME; DATETIME; TIMESTAMP; YEAR;
BOOLEAN/BOOL = TINYINT(1); JSON; ENUM (...); SET(...);

COND DECIMAL/NUMERIC are exact; FLOAT/DOUBLE approximate. UNSIGNED excludes negatives.

COND CHAR is fixed-width; VARCHAR is variable-width. TEXT stores long text; BLOB stores bytes.

Date/time functions
CURDATE()/CURRENT_DATE(); CURTIME()/CURRENT_TIME();
NOW()/CURRENT_TIMESTAMP();
UTC_TIMESTAMP();
YEAR(d); QUARTER(d); MONTH(d); DAY(d); DAYOFYEAR(d);
DAYNAME(d); MONTHNAME(d); LAST_DAY(d); DAYOFWEEK(d) Sun=1; WEEKDAY(d) Mon=0;
HOUR(t); MINUTE(t); SECOND(t); EXTRACT (unit FROM dt);
DATE(dt); TIME(dt);
DATE_ADD(d, INTERVAL n DAY); DATE_SUB(d, INTERVAL n MONTH);
TIMESTAMPADD(unit,n,dt); DATEDIFF(end, start);
TIMESTAMPDIFF(unit,start,end);
ADDTIME(t,'00:30:00'); SUBTIME(t,'00:30:00');
TIMEDIFF(end,start); TIME_TO_SEC(t); SEC_TO_TIME(n);
DATE_FORMAT(dt,'%Y-%m-%d'); STR_TO_DATE(s,'%d/%m/%Y');

COND DATEDIFF(end,start) returns days and ignores time; TIMESTAMPDIFF returns the chosen unit. INTERVAL supports date/time arithmetic.

! MySQL TIMESTAMP converts by session time zone; DATETIME preserves the stored date/time value.

03 NULL / AGGREGATION

x = NULL is wrong; use x IS NULL.
Comparisons with NULL yield UNKNOWN; WHERE keeps only TRUE. Arithmetic with NULL usually returns NULL.

Counts COUNT(*) counts output rows; COUNT(x) ignores NULL; COUNT(DISTINCT x) counts unique non-NULL values.

COND Every selected nonaggregate expression must be grouped in standard SQL. WHERE filters rows before grouping; HAVING filters groups after aggregation.

! After a LEFT JOIN, COUNT(*) still counts a NULL-extended row; COUNT(B.id) ignores it.

04 JOIN / SET / SUBQUERY

Example data A(id,name): (1,Ana),(2,Bo); B(id,tag): (1,DB),(3,AI). Match on id; output is (name,tag).

JOIN	Output (name,tag)
INNER JOIN	(Ana,DB)
CROSS JOIN	(Ana,DB), (Ana,AI), (Bo,DB), (Bo,AI)
LEFT JOIN	(Ana,DB), (Bo,NULL)
RIGHT JOIN	(Ana,DB), (NULL,AI)
FULL OUTER	(Ana,DB), (Bo,NULL), (NULL,AI)

COND Bare JOIN defaults to INNER JOIN; MySQL has no direct FULL OUTER JOIN syntax.

ON vs WHERE
A LEFT JOIN B ON A.id=B.id AND B.tag='DB'

The ON filter keeps Bo with NULL; putting B.tag='DB' in WHERE removes Bo.

Join traps NATURAL JOIN matches every same-named column, which can join unrelated fields; USING(k) names shared keys explicitly.

Anti-join: LEFT JOIN, then WHERE the right key IS NULL.

Set operations UNION / INTERSECT / EXCEPT require union-compatible columns. They remove duplicates by default; UNION ALL preserves duplicates.

Subqueries
WHERE x IN (SELECT ...)
WHERE EXISTS (SELECT 1 FROM S WHERE ...)
WHERE NOT EXISTS (...)

Correlated subqueries refer to outer-row aliases.

For “for all”, test that there is no missing requirement (e.g. required set EXCEPT possessed set is empty).

! NOT IN with any NULL in its subquery can yield UNKNOWN; NOT EXISTS is safer for anti-joins.

05 DDL / DML

CREATE / UPDATE / DELETE
CREATE TABLE R (
id INT PRIMARY KEY,
x VARCHAR(20) NOT NULL,
dept VARCHAR(20),
CHECK (...),
FOREIGN KEY (dept) REFERENCES D(dept)
);
INSERT INTO R(...) VALUES (...);
UPDATE R SET x = expr [, y = expr]
WHERE condition;
DELETE FROM R WHERE condition;

! UPDATE/DELETE without WHERE affects every row. Preview target rows with SELECT ... WHERE ... first.

COND Foreign-key actions include ON DELETE/UPDATE CASCADE, SET NULL, and RESTRICT/NO ACTION.

CTE
WITH X AS (SELECT ...)
SELECT ... FROM X;

06 RANKING / WINDOWS

For scores 100, 100, 90:

Function	100	100	90
ROW_NUMBER	1	2	3
RANK	1	1	3
DENSE_RANK	1	1	2

ROW_NUMBER is sequential and unique; ties need a deterministic tie-breaker. RANK leaves gaps after ties; DENSE_RANK does not.

! NULL scores still get a rank and tie with each other. Their position varies by DBMS and sort direction; control NULL placement or filter them first.

RANK() OVER (
PARTITION BY dept
ORDER BY score DESC)

PARTITION BY restarts within each group; ORDER BY sets window order. Ranking on grouped results happens after grouping/aggregation.

Window form
window_fn(expr) OVER (
[PARTITION BY p] [ORDER BY o]
[ROWS frame])

Windows calculate per row without collapsing rows. Aggregate windows include SUM/AVG/COUNT/MIN/MAX; ranking functions are nonaggregate.

Frames
-- Running: first row through current
SUM(x) OVER (ORDER BY d ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
-- Moving: previous 2 plus current
AVG(x) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
-- Symmetric: previous, current, next
AVG(x) OVER (ORDER BY d ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)

! UNBOUNDED PRECEDING/FOLLOWING anchors the frame at the partition start/end; finite n PRECEDING/FOLLOWING bounds move with the current row.

! ROWS counts physical rows. RANGE groups ORDER BY peers, so ties can enlarge the frame.

07 RECURSIVE CTE

Use for transitive closure, hierarchies, and paths.

WITH RECURSIVE reach(a,b) AS (
-- Anchor: base edges
SELECT a,b FROM edge
UNION
-- Recursive term extends prior rows
SELECT r.a,e.b FROM reach r
JOIN edge e ON r.b=e.a
)
SELECT * FROM reach;

The anchor seeds rows; the recursive term refers to the CTE and repeatedly extends it.

COND UNION removes duplicates and can help closure/cycle control. UNION ALL avoids deduplication and needs a safe termination strategy.

08 PROCEDURES / TRIGGERS

MySQL stored procedure
DELIMITER //
CREATE PROCEDURE p(IN p_id INT, OUT p_cnt INT)
BEGIN
DECLARE v INT DEFAULT 0;
SELECT COUNT(*) INTO v FROM T WHERE id=p_id;
SET p_cnt=v;
END//
DELIMITER ;
CALL p(7,@n);

IN receives a caller value; OUT returns one.

Declare locals/conditions/handlers before executable statements. SELECT ... INTO stores a scalar result. DELIMITER is a client command so internal semicolons stay in the procedure body.

Triggers Fire on INSERT, UPDATE, or DELETE. BEFORE validates or changes NEW; AFTER runs after success, e.g. for audit work.

CREATE TRIGGER trg BEFORE UPDATE ON T FOR EACH ROW BEGIN
IF NEW.score < 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='bad score';
END IF;
END;

Row images: INSERT uses NEW; UPDATE uses OLD and NEW; DELETE uses OLD. SIGNAL raises a user error; a trigger error aborts the invoking statement.

09 PIVOT / CUBE / ROLLUP

Conditional aggregation Rows become grouping keys; CASE values become output columns.

General PIVOT pattern
SELECT row_dimension,
AGG(CASE WHEN pivot_column = 'val1' THEN measure ELSE 0 END) AS val1,
AGG(CASE WHEN pivot_column = 'val2' THEN measure ELSE 0 END) AS val2
FROM table_name
GROUP BY row_dimension;

! PIVOT = GROUP BY rows + CASE WHEN columns + AGG cells.

Example
SELECT location,
SUM(CASE WHEN type='THEFT' THEN 1 ELSE 0 END) theft,
SUM(CASE WHEN type='BATTERY' THEN 1 ELSE 0 END) battery
FROM crime GROUP BY location;

For amounts, replace 1 with amount.

COUNT(CASE WHEN condition THEN 1 END) also counts matches because COUNT ignores NULL.

Grouping sets For dimensions (a,b,c):

- CUBE(a,b,c): every subset, 2³ = 8 sets.
- ROLLUP(a,b,c): prefix hierarchy (abc, ab, a, empty), detail to subtotals to total.
- GROUPING(a)=1 when a is omitted in a subtotal/total; use it to distinguish generated NULL from data NULL.

MySQL supports ROLLUP. CUBE/general grouping-set syntax is commonly shown with PostgreSQL in course material.

10 LAST-MINUTE TRAPS

- Window functions preserve row count; GROUP BY collapses rows.
- RANK has gaps; DENSE_RANK has none; ROW_NUMBER is unique.
- ROWS BETWEEN n PRECEDING AND CURRENT ROW covers at most n + 1 physical rows at an interior row.
- Recursive CTE: anchor first; recursive term must reach a fixpoint or stop.
- Trigger row images: INSERT=NEW, DELETE=OLD, UPDATE=both.
- GROUPING() separates subtotal NULL from a real NULL value.

SQL query check

- SCHEMA: check type/NULL/PK/FK; VARCHAR numbers sort as text.
- ROWS/JOIN: filter population before GROUP BY/rank/window; verify join key and right-table filters in ON vs WHERE.
- AGG/GRAIN: define row grain; COUNT(*) counts rows, COUNT(x)/SUM(x)/AVG(x) ignore NULL; conditional COUNT needs NULL for nonmatches.
- NULL/RANK: use IS NULL, not = NULL; MySQL <=> is NULL-safe; Nth distinct value usually needs DENSE_RANK.
- WINDOW/PHASE: WHERE filters rows, HAVING groups; check PARTITION/ORDER/frame; filter window results via CTE/subquery.
- FINAL: window ORDER differs from display ORDER; ASC is default; compare before rounding; verify aliases, column order, LIMIT.

11 DBMS / RELATIONAL MODEL

Why a DBMS? Controls redundancy/inconsistency, difficult access, isolation, buried integrity rules, partial updates, concurrency, and authorization.

Abstraction Physical: how data is stored. Logical: which data and relationships exist. View: a user-specific representation.

COND Physical data independence means changing physical storage without changing the logical schema or applications.

Schema / instance Schema is structure; instance is a snapshot of tuples. A relation is a set of tuples (order is irrelevant); a domain is a set of allowed atomic values. NULL denotes unknown/missing.

Keys Superkey uniquely identifies tuples; candidate key is a minimal superkey; primary key is the selected candidate key. A foreign key references a key in another relation; NULL is allowed only if the column permits it.

12 RELATIONAL ALGEBRA

Rows	$\sigma_{condition}(R)$
Columns	$\pi_{columns}(R)$
All pairs	$R \times S$
Matched pairs	$R \bowtie_{condition} S$
Either set	$R \cup S$
R but not S	$R - S$
Rename	$\rho_X(R)$
Assign result	$X \leftarrow E$

RA to SQL σ = WHERE; π = SELECT DISTINCT; $\bowtie$ = JOIN ... ON; $\cup$ = UNION; $-$ = EXCEPT/anti-join; ρ = alias AS.

! RA projection removes duplicates. SQL SELECT preserves them unless DISTINCT is requested.

13 PHP / HTML SYNTAX

```
PHP patterns
$x // variable
$s = $a . $b; // concatenate strings
foreach ($xs as $x) { ... }
$row["column"] // associative-array
    field
$_GET["x"]; $_POST["x"];
$conn = mysqli_connect(...);
$r = mysqli_query($conn, $sql);
$rows = mysqli_fetch_all($r, MYSQLI_ASSOC);
if ($_SERVER["REQUEST_METHOD"] === "POST")
{ ... }
```

HTML form / dropdown

```
<form method="post">
  <select name="x">
    <option value="..." ">... </option>
  </select>
</form>
```

14 HTTP / MYSQL RESULTS

Request flow Browser request → web server → PHP interpreter → MySQL. The result returns to PHP, which creates HTML/text for the browser.

Query result `mysqli_query()` returns a result handle/object, not a PHP row array. Fetch rows with `mysqli_fetch_all($r, MYSQLI_ASSOC)`.

Associative arrays `key => value`; with `MYSQLI_ASSOC`, database column names become keys, e.g. `$row["ID"]`.

GET / POST `$_GET` comes from URL query parameters; `$_POST` comes from the request body. `$_SERVER["REQUEST_METHOD"]` identifies the method received by the page.

COND Use POST for form-body submissions; reading a variable from `$_GET`/`$_POST` is separate from validating its value.